xrpl_agent_id

EXECUTIVE SUMMARY

Portable, vendor-independent identity for AI agents
XRPL native · MIT licensed

Identity & Trust · XRP Ledger (XLS-70 Credentials + XLS-40d DIDs)
v0.5.0 · WalletProvider, fee ceiling, batch anchoring · 341 tests passing · Live on XRPL Testnet

---

S_DevLabs
S_DevLabs@outlook.com

Prepared 2026-10-28

Today, agent identity is a promise. This makes it a public fact.

THE PROBLEM

AI agents are taking real actions — spending money, calling APIs, signing contracts. But they have **no public identity** and **no portable way to prove who vouched for them.** Today, agent trust is a promise made by whoever runs the platform. That trust is invisible, vendor-locked, and impossible to audit. When an agent misbehaves, there's no public trail — only the platform vendor's private logs.

THE SOLUTION

xrpl_agent_id — a Python library that gives every AI agent a public, verifiable identity on the XRP Ledger. **No new chain. No new consensus rule. No validator vote.** Just the existing XLS-70 Credential and XLS-40d DID primitives, packaged for AI developers. Anyone in the world can verify any agent's identity and credentials with a single `ledger_entry` call — no API key, no permission, no trust in a vendor's internal logs.

HOW IT WORKS

Three on-ledger transactions:

1. **Issuer** signs `CredentialCreate` — public attestation ("this agent passed audit X")
2. **Agent** signs `CredentialAccept` — opts in; acceptance itself becomes auditable on-ledger
3. **Agent** signs `DIDSet` — pins its W3C-compatible DID Document (URI-only pattern, fits the 256-byte on-ledger cap)

Verifiers query the ledger. They see the credential, the issuer, the timestamp, the URI to the full claim details. Done.

WHAT'S ALREADY BUILT (LIVE DEMO)

Subject:
did:xrpl:2:rPNGAyKowBrtpbeVjzDkHG4uJBKdV1q3cf
Issuer:
did:xrpl:2:rHDfNdLUEd7tBXZCaUuuK5wMcKzhtpNDBU

CredentialCreate: 3C03A3E9…59E7 (lgr 21123060)
CredentialAccept: 580B8CD7…243A (lgr 21123062)
DIDSet: A230469E…A3B3 (lgr 21123064)

Library state: Full Python package, MIT licensed, **v0.5.0. 341 tests passing, 6 live integration tests gated behind** `RUN_LIVE=1` (XRPL testnet; credential expiry, DID roundtrip, authorization policy). **v0.4.3 closed three silent correctness gaps:** `CREDENTIAL_EXPIRED` is now emitted when an XLS-70 credential's `expiration` is in the past; a credential is not counted as accepted until the subject signs `CredentialAccept` (`lsfAccepted`); and DID documents are validated against the DID being resolved. Ergonomic API: `AgentIdentity.from_seed()` , `issuer.issue_credential()` , `agent.has_credential()` , `resolve_did()` .

WHY XRPL

- **Both primitives already live on mainnet** — XLS-40d since 2022, XLS-70 since 2024. Zero protocol changes required.
- **3–5 second finality** — fast enough for agent-to-agent commerce
- **Sub-cent fees** — cheap enough for high-frequency agent activity
- **Native hooks for future enhancement** — credential-aware transactions, native DID resolution RPC, MPC signers are all on the roadmap without requiring forks

WHY NOW

Agent-to-agent commerce is the next frontier — LangChain, MCP, Composio are all exploding. Every agent framework today inherits the same closed-trust problem. **The window for setting the standard is open — the first credible Python library wins.**

THE ASK

Looking for three things:

1. **Validation** from the XRPL community on the credential-type taxonomy (`KYC` , `EVAL` , `OPERATOR` , `COMPLIANCE` , etc.)
2. **An early issuer partner** to run a pilot — any organization willing to issue credentials against real audit data

3. **Testnet feedback** before mainnet launch